

1. Project initialization

Follow these steps to set up the project locally:

Clone the repository

Open your command prompt at the desired project location and run:

```
git clone https://github.com/EmrysC/MedHead_PoC_systeme_d_intervention_d_urgence
```

Navigate to Resources

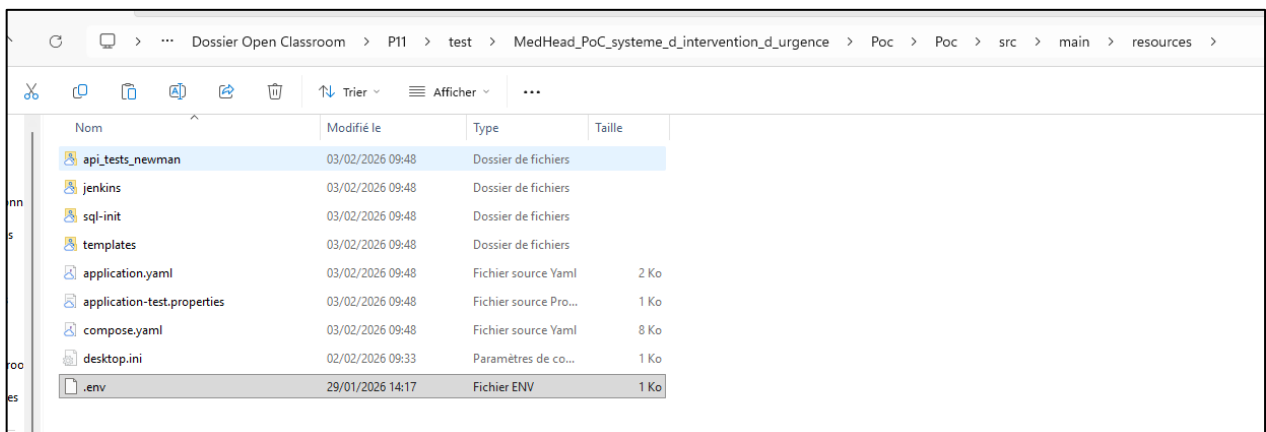
Move into the core resource directory:

```
cd "MedHead_PoC_systeme_d_intervention_d_urgence/Poc/Poc/src/main/resources"
```

Environment Variables:

Place your .env file in this directory.

Note: Environment files are hidden by default in Windows; ensure "Hidden items" is checked in your View settings to see them.



2. SonarQube setup & configuration

SonarQube is used for code quality analysis.

Launch SonarQube: Run the command

```
docker-compose up -d sonarqube
```

```
[+] Running 1/1
✓ Container sonarqube-poc Running
PS G:\Mon Drive\Formation\Open Classroom\Dossier Open Classroom\c\Poc\src\main\resources>
```

Access the UI: Open <http://localhost:8084> in your browser.

Initial Login: Use admin/admin. You will be prompted to change your password upon first login.

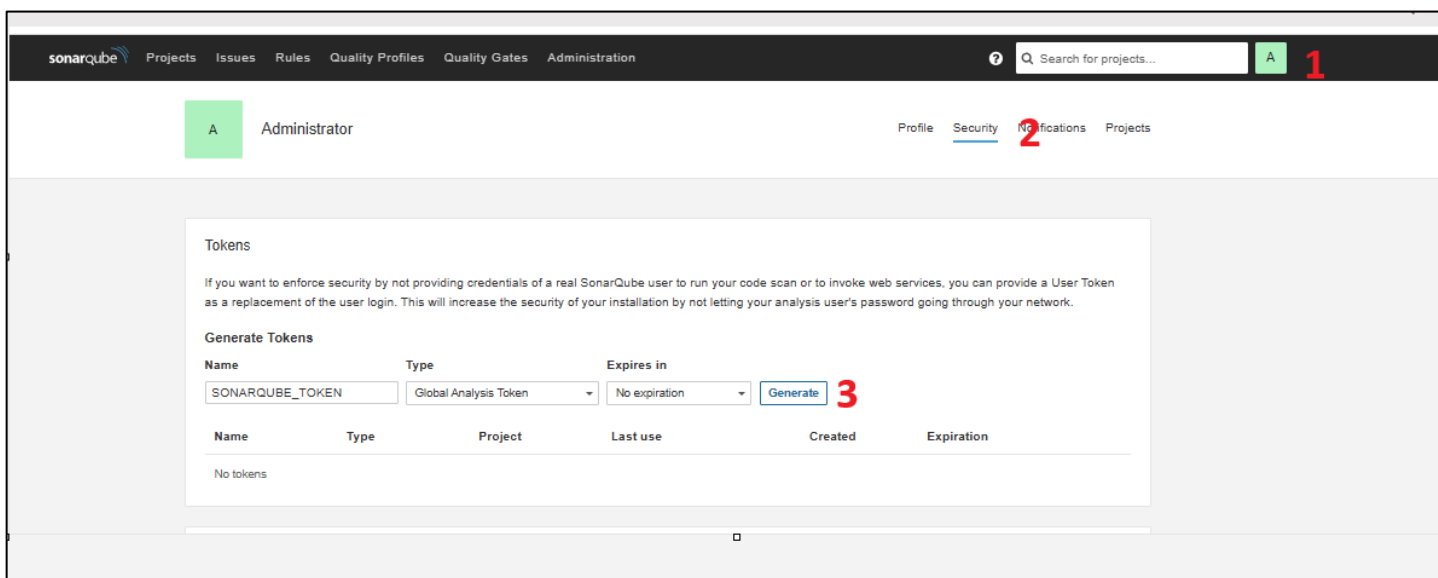
Generate Token:

Click on the A (Administrator) icon in the top right.

Select Security.

Generate a new token named

```
SONARQUBE_TOKEN
```



Copy and save this token immediately, as it will not be shown again.

New token "SONARQUBE_TOKEN" has been created. Make sure you copy it now, you won't be able to see it again!

Copy

sqa_f896a29a26af0b6f90250a10afdda32c12039995

Name	Type	Project	Last use	Created	Expiration	
SONARQUBE_T...	Global		Never	February 3, 2026	—	Revoke

Create Webhook:

Navigate to Administration > Configuration > Webhooks.

Create a webhook named Jenkins-Webhook with the URL: http://jenkins_poc:8080/sonarqube-webhook/

sonarqube

ProjectsIssuesRulesQuality ProfilesQuality GatesAdministration

Search for projects

Administration

ConfigurationSecurityProjectsSystemMarketplace

Webhooks

No webhook defined.

Create

Create Webhook

All fields marked with * are required

Name *

Jenkins-Webhook

URL *

http://jenkins_poc:8080/sonarqube-webhook/

Secret

Create

3. Jenkins Configuration

Jenkins handles the CI/CD pipeline.

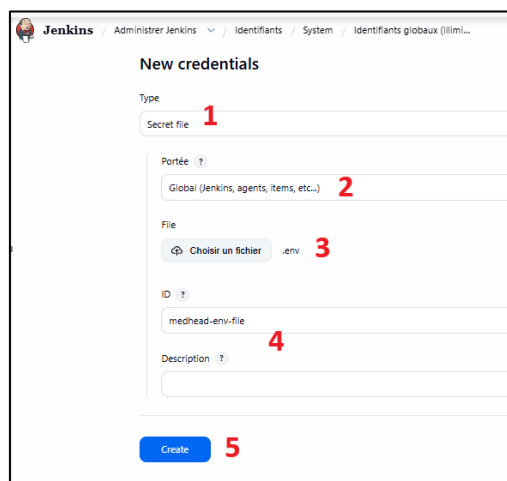
Launch Jenkins: Run

```
docker-compose up -d jenkins
```

```
PS G:\Mon Drive\Formation\Open Classroom\Dossier Open Classroom\P11\test\MedHead_PoC_systeme_
c\Poc\src\main\resources> docker-compose up -d jenkins
[+] Running 1/1
✓ Container jenkins_poc Running
PS G:\Mon Drive\Formation\Open Classroom\Dossier Open Classroom\P11\test\MedHead_PoC_systeme_
c\Poc\src\main\resources>
```

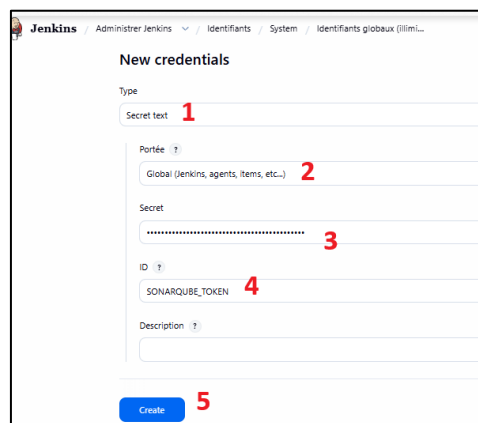
Add credentials: Go to <http://localhost:8082/manage/credentials/>.

Env File: Add a new credential of type Secret file. Set the ID to medhead-env-file and upload your .env file.



The screenshot shows the Jenkins 'New credentials' form. The 'Type' is set to 'Secret file' (1). The 'Portée' (Scope) is set to 'Global (Jenkins, agents, items, etc...)' (2). The 'File' section shows a button to 'Choisir un fichier' (Choose a file) and the file name '.env' (3). The 'ID' is set to 'medhead-env-file' (4). The 'Description' field is empty. The 'Create' button is at the bottom (5).

Sonar Token: Add a credential of type Secret text. Set the ID to SONARQUBE_TOKEN and paste your SonarQube token into the Secret field.



The screenshot shows the Jenkins 'New credentials' form. The 'Type' is set to 'Secret text' (1). The 'Portée' (Scope) is set to 'Global (Jenkins, agents, items, etc...)' (2). The 'Secret' field contains a masked token (3). The 'ID' is set to 'SONARQUBE_TOKEN' (4). The 'Description' field is empty. The 'Create' button is at the bottom (5).

Configure System: Go to Manage Jenkins > System. <http://localhost:8082/manage/configure>

Add a SonarQube installation under SonarQube servers.

Name: sonarqube-poc.

Server URL: http://sonarqube:9000.

Server authentication token: Select the SONARQUBE_TOKEN credential.

The screenshot shows the Jenkins 'Configure System' page. The browser address bar is 'localhost:8082/manage/configure'. The page title is 'Jenkins / Administrer Jenkins / System'. The 'Variables d'environnement' section is collapsed. The 'SonarQube servers' section is expanded and highlighted with a red '1'. Below it, there is a checkbox for 'Environment variables' and a section for 'Installations de SonarQube'. The 'Liste des installations de SonarQube' table is empty. A new installation form is visible with the following fields: 'Nom' (Name) with the value 'sonarqube-poc' (highlighted with a red '2'), 'URL du serveur' (Server URL) with the value 'http://sonarqube:9000' (highlighted with a red '3'), and 'Server authentication token' with the value 'SONARQUBE_TOKEN' (highlighted with a red '4'). There is an 'Avancé' (Advanced) dropdown menu. At the bottom, there is a '+ Ajouter une installation SonarQube' button and a 'Save' button (highlighted with a red '5') followed by an 'Appliquer' (Apply) button.

4. Pipeline Project Setup

Configure the specific pipeline for this project at <http://localhost:8082/job/MedHead-Pipeline/configure>.

Definition: Select Pipeline script from SCM.

SCM: Select Git.

Repository URL: https://github.com/EmrysC/MedHead_PoC_systeme_d_intervention_d_urgence

Branch Specifier: Set to */main.

Script Path: Set to Poc/Poc/src/main/resources/jenkins/Jenkinsfile

The screenshot shows the Jenkins 'Configure' page for a pipeline named 'MedHead-Pipeline'. The left sidebar has tabs for 'Général', 'Triggers', 'Pipeline', and 'Advanced', with 'Pipeline' selected. The main area is titled 'Configurer' and contains the following fields:

- Pipeline**: A section header with the text 'Define your Pipeline using Groovy directly or pull it from source control.'
- Definition**: A text field containing 'Pipeline script from SCM' with a red '1' next to it.
- SCM**: A dropdown menu with 'Git' selected, marked with a red '2'.
- Repositories**: A section containing:
 - Repository URL**: A text field with the value 'https://github.com/EmrysC/MedHead_PoC_systeme_d_intervention_d_urgence.git', marked with a red '3'.
 - Credentials**: A dropdown menu with '- aucun -' selected.
 - Avancé**: A dropdown menu.
 - + Add Repository**: A button.
- Branches to build**: A section containing:
 - Branch Specifier (blank for 'any')**: A text field with the value '*/main', marked with a red '4'.
 - + Add Branch**: A button.
- Navigateur de la base de code**: A dropdown menu with '(Auto)' selected.
- Additional Behaviours**: A section with a '+ Ajouter' button.
- Script Path**: A text field with the value 'Poc/Poc/src/main/resources/jenkins/Jenkinsfile', marked with a red '5'.

At the bottom, there are two buttons: 'Save' and 'Appliquer', with a red '6' next to 'Appliquer'.

5. Execution

Once configured, you can run and monitor the pipeline.

Open Blue Ocean: Navigate to <http://localhost:8082/blue/organizations/jenkins/MedHead-Pipeline/activity>.

Run Pipeline: Click Lancer (Run) to start the build.

Monitor Progress: Follow the stages from Initialization through Analysis and End.

The screenshot displays the Jenkins Blue Ocean interface for the 'MedHead-Pipeline'. The top navigation bar shows the pipeline name and a 'Lancer' button, which is highlighted with a red '1'. A modal dialog in the center states 'This job has not been run' with a 'Lancer' button. The bottom section shows the pipeline progress bar with stages: Start, Initialisation Workspace, Compilation & Tests, Nettoyage & Préparation, Lancement Application, Analyse (SonarQube), Set jeu de données pour te..., Tests API (Newman), Tests E2E (Selenium), and End. The 'Analyse (SonarQube)' stage is currently selected.